

ABACUS_PROJECT_AUDIT.md — Auditoria Completa do Estado Atual

Documento crítico e honesto. Esta é a avaliação consolidada do estado real do projeto **Born to Run — Treinamento e Saúde**, baseada na auditoria técnica bruta (`AUDIT_RAW_FINDINGS.md`) e na inspeção visual (`VISUAL_INSPECTION_REPORT.md`). O objetivo não é agradar, e sim expor com precisão o que existe, o que está quebrado e o que precisa ser reconstruído. Nenhum problema foi suavizado.

Item	Valor
Data da auditoria	17/07/2026
Branch analisada	main (auditoria) / abacus-fable-rebuild (documentação)
Stack real	Next.js 16.2.10, React 19.2.4, TypeScript, Tailwind v4, Supabase
Arquivos-fonte	77 (excluindo <code>node_modules</code> , <code>.next</code> , <code>.git</code>)
Histórico Git	2 commits (um “big commit” único, sem histórico incremental)
Veredito geral	Não deployável. Build falha; funcionalidades centrais quebradas; visual parcialmente profissional com falhas graves de conteúdo.

1. Veredito Executivo

O projeto tem uma **fachada convincente** (a home pública é bonita e profissional) sobre uma **base estruturalmente comprometida**. Por trás da landing page existe um sistema que **não compila**, com **duas implementações concorrentes** da mesma funcionalidade — uma quebrada e outra correta — e uma **documentação de entrega (`HANDOVER.md`) que descreve um produto diferente do que foi realmente construído**.

Em termos práticos:

- **Não é possível fazer deploy** — o `npm run build` falha em erro de tipo.
- **As funcionalidades centrais da área logada não funcionam:** feed, criação de posts, curtidas, comentários, criação de treino por uma das vias, página de fotos, logout do aluno e recuperação de senha estão quebrados.
- **O design público engana:** a home parece pronta, mas a galeria exibe fotos de banco de imagens totalmente fora de contexto (Torre Eiffel, panquecas, natação) e imagens quebradas, e todos os contatos são placeholders falsos.

- **A navegação não funciona como site multipágina** — usa âncoras que só valem na home.

A boa notícia: **a fundação de dados e segurança (schema + RLS) é sólida e bem modelada**, e existe um conjunto **correto** de server actions (`lib/actions/`) que pode ser aproveitado. A reconstrução é viável e vale a pena, desde que se comece consolidando o que existe antes de adicionar qualquer coisa nova.

2. Pontos Positivos (o que preservar)

Ser crítico não significa ignorar o que está bom. Estes ativos devem ser **preservados** na reconstrução:

1. **Schema de banco bem modelado** (`supabase/schema.sql`): tabelas coesas (`profiles` , `posts` , `comments` , `likes` , `workouts` , `announcements`), índices adequados, triggers de `updated_at` , trigger `handle_new_user` para criação automática de perfil e função `is_admin()` com `SECURITY DEFINER` .
2. **RLS habilitado em todas as tabelas** com policies em sua maioria corretas — uma base de segurança real, não decorativa.
3. **Server actions corretas em `lib/actions/`** (`feed.ts` , `admin.ts` , `auth.ts`): alinhadas ao schema, com upload de foto, toggle de like, comentários e CRUD administrativo funcionando conceitualmente.
4. **Autorização admin server-side** no `app/(admin)/layout.tsx` : busca o perfil e checa `role === 'admin'` , redirecionando quem não é — correto e seguro.
5. **Setup do Supabase SSR** (`lib/supabase/client.ts` e `server.ts`) correto para o padrão de cookies do App Router.
6. **Identidade visual da home**: paleta consistente (off-white `#F9F7F5` , vermelho `#DC2626` , texto `#1C1917`), tipografia profissional (Inter + Barlow Condensed), fotos reais da equipe, layout split-screen elegante nas telas de auth.
7. **Responsividade sólida** na maior parte das páginas públicas (mobile 375px, tablet 768px, desktop 1440px).
8. **PWA configurado** (`manifest.json` válido, ícones 192/512, atalhos).
9. **Tokens de tema `--color-btr-*`** já definidos no `globals.css` — um ponto de partida para o design system.

3. Problemas Técnicos

3.1 Build, lint e typecheck falham (bloqueante)

- `npm run build` **FALHA (exit 1)**. Compila, mas quebra no type-check com `TS2613` : `components/feed/PostCard` **não tem default export**, mas `app/(dashboard)/dashboard/feed/page.tsx` o importa como default. **O projeto não é deployável**. Como o build para no primeiro erro, podem existir mais erros de tipo ocultos após esse ponto.
- `npm run lint` **FALHA (exit 1)**: 12 erros + 12 warnings. Erros de `@typescript-eslint/no-explicit-any` e `react/no-unescaped-entities` ; warnings de imports/variáveis não usados.
- `npm run tsc --noEmit` **FALHA (exit 2)**: o mesmo `TS2613` . Só aparece 1 erro porque o cliente Supabase é usado **sem tipos Database gerados**, então queries com colunas erradas retornam

tipos fracos (`any` / `unknown`) e **não** falham em compile-time — falham só em runtime, mascarando dezenas de bugs.

- **Aviso do Next 16:** `middleware.ts` usa convenção **descontinuada** (deveria migrar para `proxy`).

3.2 Duplicação estrutural (o problema mais grave)

O repositório contém **implementações concorrentes e conflitantes** da mesma coisa. Isto sugere fortemente que o código é resultado de **duas gerações sobrepostas** cuja fusão nunca foi concluída.

- **Dois conjuntos de server actions:**
- `app/actions/` (`post.ts` , `profile.ts` , `workouts.ts`) → **QUEBRADO** (usa colunas/papéis inexistentes).
- `lib/actions/` (`feed.ts` , `admin.ts` , `auth.ts`) → **CORRETO** (alinhado ao schema).
- **Dois middlewares:** `middleware.ts` (raiz, ativo) e `lib/supabase/middleware.ts` (`updateSession` , **nunca importado — código morto**).
- **Rotas duplicadas:** `/dashboard` vs `/dashboard/feed` ; `/treinos` vs `/dashboard/treinos` ; `/perfil` vs `/dashboard/perfil` . **A sidebar aponta para a metade quebrada.**
- **Componentes concorrentes:** `CreatePost` vs `NewPostForm` ; `ProfileForm` vs `PerfilForm` .

3.3 Bugs de runtime por colunas/relacionamentos inexistentes

- `Feed /dashboard` : query usa FK inexistente `posts_created_by_fkey` e lê `content / image_url` (colunas que **não existem** — o certo é `caption / photo_url`).
- `/fotos` : consulta `image_url` (inexistente).
- `/treinos` : usa FK inexistente `workouts_created_by_fkey` e checa papel `'treinador'` (impossível no schema).
- `app/actions/post.ts` : insere `content / image_url / created_by` (inexistentes) → falha.
- `app/actions/workouts.ts` : insere `assigned_to` (inexistente) e checa `'treinador'` → falha.

3.4 Estilização quebrada em larga escala

- Conflito **Tailwind v3 vs v4:** `tailwind.config.ts` está em formato v3 e **não é carregado** (não há `@config` no CSS). É código morto.
- Dezenas de **classes utilitárias custom indefinidas** usadas nas páginas: `card` , `btn-primary` , `btn-secondary` , `btn-outline` , `badge` , `badge-red/green/orange/gray` , `section-title` , `section-subtitle` , `input-base` , `divider-*` , `shadow-card-lg` . Também variáveis como `var(--color-red)` , `var(--color-green)` não existem (o que existe é `--color-btr-*`).
- **Páginas afetadas:** `sobre` , `equipe` , `galeria` , `resultados` , `contato` , `layout/páginas de (auth)` secundárias, `dashboard/treinos` , `dashboard/perfil` , `dashboard/feed` , **todas** as de `(admin)` , e os formulários `NewPostForm` , `PerfilForm` , `AdminForm` . A home escapa porque usa hex hardcoded.
- **Fontes inconsistentes:** `layout.tsx` carrega Inter + Barlow Condensed; `tailwind.config.ts` cita `Outfit` ; `globals.css` cita Barlow Condensed. Sem alinhamento.

3.5 Autenticação com pontas soltas

- **Logout do aluno quebrado:** `(dashboard)/layout.tsx` faz POST para `/auth/signout` , rota que **não existe**. A action `logout()` correta existe em `lib/actions/auth.ts` , mas não é usada ali.
- **Recuperação de senha quebrada:** `resetPassword` aponta `redirectTo` para o domínio do **Supabase** (não do app) e para a rota `/recuperar-senha/nova` , que **não existe**.
- **Links mortos** no login: “Esqueceu a senha?” aponta para `#` ; “Lembrar de mim” não conectado a nada.
- **Duplicação de lógica de auth:** `login/cadastro` usam o client Supabase diretamente, ignorando as actions de `lib/actions/auth.ts` .

3.6 Ausência de padrões do App Router

- **Sem `error.tsx`, `not-found.tsx` ou `loading.tsx`** em nenhum nível → sem boundaries de erro, sem estados de carregamento padronizados, 404 genérico do Next.
- **Sem validação de schema** (ex.: zod) no client ou nas actions; validações são só `required` de HTML.
- **Tratamento de erro inconsistente:** várias queries ignoram o objeto `error` do Supabase; curtidas/comentários “To implement” com valores fixos.

4. Problemas Visuais

4.1 Críticos

1. **Galeria com imagens totalmente fora de contexto:** fotos de banco de imagens de **Torre Eiffel** (rotulada “Grupo de atletas unidos”), **panquecas com frutas** (rotulada “Equipe celebrando juntos”), **natação em piscina** e **academia de musculação** — nada disso é corrida de rua. **Destrói a credibilidade.**
2. **3 imagens quebradas na galeria** (URLs do Unsplash retornam `naturalWidth: 0`) → ícone de imagem quebrada.
3. **Contatos placeholder falsos:** dois números de WhatsApp diferentes (`5519900000000` e `5519999999999`) e e-mails fictícios. Um cliente não conseguiria ser contatado.
4. **Navegação por âncoras (`#sobre` , `#modalidades`)** funciona só na home. Em `/galeria` , clicar “Sobre” leva a `/galeria#sobre` , que não existe → site inutilizável como multipágina.

4.2 Moderados

1. **Páginas órfãs:** `/sobre` , `/equipe` , `/galeria` , `/resultados` , `/contato` existem mas **não há links diretos** para elas no menu — o usuário não as descobre.
2. **CTA “Comece Agora” inconsistente:** no header vai para `/cadastro` ; no footer vai para `#modalidades` .
3. **Rota `/historia` retorna 404** (a história está embutida em `/sobre`).
4. **H1 da home sem espaço** entre palavras para leitores de tela (`<br>` cola “transformação-começa”).
5. **Contraste insuficiente** nas estatísticas de `/resultados` (vermelho sobre bege claro) — risco de reprovar WCAG AA.

4.3 Menores

1. `/dashboard/comunicados` retorna 404 (não existe no filesystem).
2. Link do Instagram (`instagram.com/equipeborntorun`) precisa ser confirmado.
3. Assets padrão do `create-next-app` (`next.svg` , `vercel.svg` , etc.) não foram removidos.

5. Funcionalidades: Reais vs. Simuladas vs. Ausentes

✓ Reais (conectadas ao Supabase e alinhadas ao schema)

- Cadastro e login (via client Supabase nas páginas de auth).
- Trigger de criação automática de perfil no cadastro.
- Autorização admin server-side (`(admin)/layout.tsx`).

- CRUD admin de treinos e comunicados (`lib/actions/admin.ts`).
- Atualização de perfil e upload de avatar (bucket `avatars`).
- Leitura de treinos em `/dashboard/treinos` (sem privacidade por aluno).
- Proteção de rotas pelo middleware (redirect para login).

● Simuladas / Estáticas (hardcoded)

- Todo o conteúdo das páginas públicas: números (“500+ atletas”, “98%” — que **não** batem com os dados oficiais), depoimentos, textos.
- Galeria: fotos de Unsplash hardcoded (com URLs quebradas e irrelevantes).
- Formulário de contato: sem backend confirmado (não envia para lugar nenhum verificável).
- Curtidas/comentários no feed principal: valores fixos em 0, com comentário “To implement”.

✗ Ausentes / Quebradas

- Build/deploy (falha de tipo).
- Feed principal `/dashboard` (colunas/FK erradas).
- `/dashboard/feed` (import quebrado do PostCard).
- Criar post via `app/actions/post.ts` .
- Curtir e comentar (botões sem handler; actions corretas existem mas estão mortas).
- Criar treino via `app/actions/workouts.ts` .
- `/fotos` (coluna inexistente).
- Logout do aluno (rota inexistente).
- Recuperação de senha (redirect/rota errados).
- Remover membro (sem policy DELETE em `profiles`).
- Privacidade de treino por aluno (não existe no schema; RLS mostra tudo a todos).
- Tela de comunicados dedicada para o aluno.
- Estados de erro/loading/404 customizados.

6. Código Ruim / Dívida Técnica

- **Duplicação em três camadas** (actions, rotas, componentes) — o maior débito. Cada feature tem uma versão certa e uma errada convivendo.
- **Código morto:** `lib/supabase/middleware.ts` , actions de `app/actions/` , `lib/actions/feed.ts` (não chamadas por nenhum componente), `logout()` (não usada onde precisaria), `tailwind.config.ts` .
- **any disseminado** — mascara erros de tipo.
- **Cliente Supabase sem tipos Database** — a maior fonte de bugs silenciosos.
- `types/index.ts` **diverge do HANDOVER** (não tem `treinador`) — o correto, mas o código em `app/actions` ainda usa `'treinador'` .
- **Sem camada de validação** (zod ou similar).
- **Sem testes** de nenhum tipo (unitário, integração, e2e).
- **Documentação divergente e enganosa** (`HANDOVER.md`), que induziria o próximo dev ao erro.
- **Telefones e e-mails placeholder** espalhados e inconsistentes.
- **Histórico Git inútil** (um único “big commit”), impossibilitando bisect/rastreio de decisões.

7. Riscos e Vulnerabilidades

Risco	Descrição	Severidade
Build quebrado	Impossível fazer deploy; qualquer CI falha	● Crítico
Bugs de runtime silenciosos	Sem tipos Database , erros de coluna só aparecem em produção	● Crítico
Falha silenciosa de exclusão	<code>deleteMember</code> não avisa que a RLS bloqueou (falta policy DELETE)	● Alto
<code>profiles_update_admin</code> sem <code>WITH CHECK</code>	UPDATE de admin pode gravar valores fora do escopo pretendido	● Alto
Privacidade de treino inexistente	RLS <code>USING (true)</code> expõe todos os treinos a todos os autenticados (contradiz a UI)	● Alto
Middleware sem reforço de admin	Proteção admin depende só do layout; sem defesa em profundidade	● Médio
Convenção <code>middleware</code> descontinuada	Quebra futura no Next 16+	● Médio
2 vulnerabilidades moderadas	<code>postcss</code> transitivo via <code>next</code> (<code>npm audit</code>)	● Médio
Credibilidade / reputação	Imagens irrelevantes e contatos falsos publicados prejudicam um cliente real	● Alto (negócio)
Dados inventados	Estatísticas hardcoded conflitam com os dados oficiais	● Médio (confiança)

Nota sobre segredos: a auditoria **não** encontrou chaves reais versionadas (o `.env.local` presente contém apenas placeholders). Ainda assim, `.env*` está corretamente no `.gitignore` e um `.env.example` deve ser mantido.

8. O Que Pode Ser Removido

- `app/actions/post.ts`, `app/actions/workouts.ts`, `app/actions/profile.ts` (após migrar o que for útil para `lib/actions/`).
- `lib/supabase/middleware.ts` (código morto).
- Rotas duplicadas quebradas: `app/(dashboard)/dashboard/page.tsx` (feed quebrado), `app/(dashboard)/treinos/page.tsx`, `app/(dashboard)/perfil/page.tsx`, `app/(dashboard)/fotos/page.tsx` — **após** consolidar tudo sob `/dashboard/*`.
- Componentes concorrentes redundantes: escolher entre `CreatePost / NewPostForm` e entre `ProfileForm / PerfilForm`, removendo o perdedor.
- `tailwind.config.ts` (se optar 100% pela config v4 no CSS) — ou reativá-lo com `@config` (decisão única).
- Assets padrão do create-next-app em `public/` (`next.svg`, `vercel.svg`, `window.svg`, `globe.svg`, `file.svg`).
- Imports/variáveis não usados (12 warnings de lint).

9. O Que Pode Ser Preservado

- `supabase/schema.sql` — fonte da verdade do banco (com pequenos ajustes de RLS na Fase 2).
- `lib/actions/` — base correta de server actions.
- `lib/supabase/client.ts` e `server.ts` — setup SSR correto.
- `app/(admin)/layout.tsx` — autorização admin correta.
- **Tokens** `--color-btr-*` do `globals.css` — semente do design system.
- **Fotos reais** em `public/` (`logo.png`, `robson-portrait.jpg`, `robson-running.jpg`, `team-group.jpg`).
- **Estrutura de route groups** (`(public)`, `(auth)`, `(dashboard)`, `(admin)`) — boa organização.
- `manifest.json` — PWA já configurado.
- **Home pública** — reaproveitável após alinhar tokens e corrigir navegação/CTAs.

10. Conclusão e Recomendação

O Born to Run **não está pronto para produção** e **não deve ser deployado** no estado atual. Porém, **não é um caso de recomeçar do zero**: a modelagem de dados e a camada de segurança são bons ativos, e há um conjunto correto de server actions para ancorar a reconstrução.

Recomendação: executar o `FABLE_REBUILD_PLAN.md` na ordem, começando pela **consolidação** (eliminar duplicação, escolher a metade correta, fazer o build passar) e pela **fundação de dados/segurança** (corrigir RLS, decidir modelo de papéis e privacidade), antes de reconstruir design e features. A prioridade zero é transformar um projeto que **não compila** em um projeto que **compila, é honesto no conteúdo e seguro por padrão**.

Fim do ABACUS_PROJECT_AUDIT.md — auditoria consolidada. Nenhum código-fonte foi alterado.